Decroly
SANTANDER

UD7

DESARROLLO DE INTERFACES GRÁFICAS

MP_0485
Programación

7.1 Introducción a Java FX

Introducción

Las interfaces gráficas de aplicaciones informáticas, conocidas como GUI (Graphical User Interface), son los medios visuales mediante los cuales los usuarios interactúan con programas de software. Estas interfaces reemplazan las antiguas líneas de comandos con elementos gráficos como ventanas, íconos, menús y botones, haciendo que el uso de las aplicaciones sea más accesible e intuitivo, incluso para personas sin conocimientos técnicos.

Los componentes principales de una GUI:
Ventanas: áreas rectangulares donde se presenta la información y se permite la interacción con la aplicación.

- Botones: permiten al usuario realizar acciones con un solo clic.
- Menús: listas desplegables que agrupan funciones relacionadas.
- Cajas de texto: áreas donde el usuario puede ingresar información.
- Etiquetas: textos que describen u orientan al usuario sobre otros elementos de la interfaz.
- Iconos: imágenes pequeñas que representan funciones, archivos o accesos directos.
- Barras de herramientas: conjuntos de botones o controles agrupados para acceso rápido.

Las interfaces gráficas nos ofrecen unas ventajas respecto de las antiguas experiencias de usuario:

- Facilidad de uso: reducen la necesidad de memorizar comandos.
- Accesibilidad: permiten a más personas interactuar con tecnología.
- Aprendizaje más rápido: los elementos visuales ayudan a entender mejor el funcionamiento de la aplicación.
- Interactividad: ofrecen una experiencia más rica y dinámica.

Dependiendo del lenguaje de programación y el entorno, se pueden usar distintas herramientas:

- Java: Swing, JavaFX.
- Python: Tkinter, PyQt, Kivy.
- C#: Windows Forms, WPF.
- JavaScript (para aplicaciones web): React, Vue, Angular.
- Kotlin: Jetpack Compose.

Actualmente las interfaces gracias se rigen por unas tendencias o patrones generales:

- Diseño responsivo: interfaces que se adaptan a distintos tamaños de pantalla.
- Dark mode: opciones de visualización con colores oscuros.
- Accesibilidad: diseño que toma en cuenta usuarios con discapacidades.
- Interfaces táctiles y por voz: más allá del mouse y teclado.

Java FX

JavaFX es una plataforma para el desarrollo de interfaces gráficas de usuario (GUI) en aplicaciones Java. Fue introducida como sucesora de Swing con el objetivo de ofrecer una experiencia más moderna, rica y flexible para aplicaciones de escritorio (y también móviles, en algunos casos). JavaFX fue desarrollado originalmente por Sun Microsystems (la misma compañía que creó Java), con su primera versión lanzada en 2008. Su propósito inicial era competir con otras tecnologías de interfaces ricas como Adobe Flash, Microsoft Silverlight, y Java Applets, ofreciendo una manera moderna de crear interfaces ricas en medios (multimedia, gráficos, animaciones) dentro del entorno Java.

Línea de tiempo clave:

- 2008 (JavaFX 1.0): Primera versión pública. Se usaba un lenguaje propio llamado JavaFX Script, separado de Java.

- 2010 (Oracle compra Sun Microsystems): JavaFX pasa a ser desarrollado por Oracle.

- 2011 (JavaFX 2.0): Se elimina JavaFX Script y se reescribe completamente en Java puro, lo que facilitó su adopción.

- 2014 (Java 8): JavaFX se integra en el JDK por defecto, coexistiendo con Swing.

- 2018 (Java 11): JavaFX se separa del JDK y pasa a desarrollarse como un proyecto independiente: OpenJFX.

Actualidad: Se mantiene como proyecto de código abierto (https://openjfx.io), con soporte activo por parte de la comunidad y empresas como Gluon, que además impulsa JavaFX en dispositivos móviles. Desde Java 11, JavaFX ya no está incluido por defecto en el JDK, pero puede añadirse fácilmente como una biblioteca externa, por ejemplo, usando Maven, Gradle, o descargando el SDK de JavaFX.

Características principales de JavaFX

- Basado en Java: Se integra completamente con el lenguaje Java, lo que facilita su uso para desarrolladores ya familiarizados con el ecosistema Java.

- Sistema de Escena (Scene Graph): JavaFX organiza la interfaz en una estructura jerárquica llamada Scene Graph, donde cada nodo representa un componente visual (botón, texto, imagen, etc.).

- FXML (Lenguaje declarativo): Permite definir la interfaz de usuario usando XML en lugar de Java puro, separando claramente la lógica del diseño visual.

- CSS para estilos: JavaFX permite usar hojas de estilo en cascada (CSS) para aplicar estilos visuales a la interfaz, similar al desarrollo web.

- Soporte multimedia: Incorpora soporte para audio, video y gráficos 2D y 3D, ideal para aplicaciones interactivas o visuales.

- Animaciones y efectos gráficos: Facilita la creación de animaciones (transiciones, rotaciones, desvanecimientos) y efectos visuales (sombras, reflejos, desenfoques).

- Bindings (Enlace de propiedades): Permite que las propiedades de los elementos estén conectadas entre sí, facilitando la sincronización automática de datos y comportamientos.

- Interfaz responsiva: Ofrece herramientas para diseñar interfaces que se adaptan dinámicamente al tamaño de la ventana.

- Multiplataforma: Funciona en Windows, macOS y Linux. También ha sido usada experimentalmente para desarrollo móvil mediante frameworks como Gluon.

- Integración con herramientas de desarrollo: Se puede usar con entornos como IntelliJ IDEA, NetBeans o Eclipse, y con herramientas visuales como Scene Builder, que permite diseñar interfaces arrastrando y soltando componentes.

Herramientas y entornos de desarrollo

Cuando se trabaja con JavaFX, elegir las herramientas y entornos adecuados puede facilitar enormemente el desarrollo, el diseño visual y la organización del proyecto. Aquí te explico las principales herramientas recomendadas y sus funciones dentro del flujo de trabajo con JavaFX.

Entornos de desarrollo:

Los IDEs te permiten escribir, depurar y compilar código de forma eficiente. Algunos ofrecen soporte específico para JavaFX.

IntelliJ IDEA

- Muy popular entre desarrolladores Java.
- Buena integración con Maven y Gradle.
- Compatible con JavaFX a través de configuración manual o plantillas.
- Puedes usar Scene Builder integrándolo o abriéndolo externamente.

NetBeans

- IDE originalmente desarrollado por Sun/Oracle, muy amigable con JavaFX.
- Soporta FXML y Scene Builder de forma nativa.
- Fácil de configurar para proyectos JavaFX sin necesidad de añadir librerías externas (especialmente en versiones anteriores a Java 11).
- Permite previsualizar la UI mientras editas el código.

Eclipse:

- Ampliamente usado, pero requiere algo más de configuración para trabajar con JavaFX.
- Necesita plugins externos como e(fx)clipse para ofrecer soporte completo.
- A partir de Java 11, requiere configurar manualmente las rutas del módulo JavaFX.

Scene Builder (Gluon)

- Herramienta visual para diseñar interfaces gráficas usando FXML.
- Permite crear GUIs con arrastrar y soltar sin escribir código Java.
- El diseño se guarda en archivos .fxml que luego se enlazan a controladores Java.
- Muy útil para diseñadores y desarrolladores que prefieren separar UI y lógica.

Puedes descargarlo aquí: https://gluonhq.com/products/scene-builder/

Gestores de dependencias

JavaFX a partir de Java 11 ya no está incluido en el JDK, así que necesitas añadirlo manualmente:

Maven: Sistema de gestión de proyectos que permite declarar dependencias en un archivo pom.xml. Muy usado en proyectos Java empresariales.

Añades JavaFX así:

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>20.0.0</version>
</dependency>
```

Gradle: Alternativa más moderna y ligera a Maven. Se define la dependencia en build.gradle:

```
dependencies {
    implementation 'org.openjfx:javafx-controls:20.0.0'
}
```

Ambos permiten automatizar la compilación, empaquetado y ejecución del proyecto con JavaFX.

Pruebas y depuración

JavaFX es compatible con frameworks de pruebas como JUnit para lógica de negocio. Para pruebas de GUI, se pueden usar herramientas como TestFX: framework para pruebas automatizadas de interfaces JavaFX.

Arquitectura de JavaFX

JavaFX proporciona una API completa con un rico conjunto de clases e interfaces para crear aplicaciones GUI con gráficos enriquecidos. Los paquetes importantes de esta API son:

- javafx.animation: Contiene clases para añadir animaciones basadas en transiciones, como relleno, desvanecimiento, rotación, escala y traslación, a los nodos JavaFX.
- javafx.application: Contiene un conjunto de clases responsables del ciclo de vida de la aplicación JavaFX.
- javafx.css: Contiene clases para añadir estilos tipo CSS a las aplicaciones GUI JavaFX. javafx.event: Contiene clases e interfaces para entregar y manejar eventos JavaFX. javafx.geometry: Contiene clases para definir objetos 2D y realizar operaciones sobre

ellos. javafx.stage: Este paquete contiene las clases contenedoras de nivel superior para la aplicación JavaFX. javafx.scene - Este paquete proporciona clases e interfaces para soportar el gráfico de escena. Además, también proporciona subpaquetes como canvas, chart, control, effect, image, input, layout, media, paint, shape, text, transform, web, etc. Existen varios componentes que soportan esta rica API de JavaFX.

La siguiente figura muestra la arquitectura de la API JavaFX. Aquí puede ver los componentes compatibles con la API JavaFX.

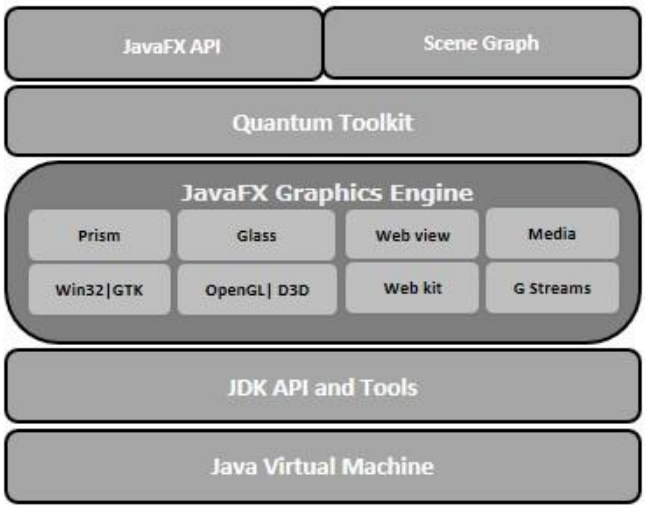

Figura 1: Arquitectura de Java FX. Fuente: tutorialspoint.com.

En JavaFX, las aplicaciones GUI se programan utilizando un Scene Graph. Un Scene Graph es el punto de partida de la construcción de la aplicación GUI. Contiene las primitivas de la aplicación (GUI) que se denominan nodos. Un nodo es un objeto visual/gráfico y puede incluir:

- Objetos geométricos (gráficos) - (2D y 3D) como círculos, rectángulos, polígonos, etc.
- Controles de interfaz de usuario, como botones, casillas de verificación, cuadros de selección, áreas de texto, etc.
- Contenedores (paneles de diseño): panel de bordes, panel de cuadrícula, panel de flujo, etc.
- Elementos multimedia: objetos de audio, vídeo e imagen.

En general, una colección de nodos constituye un grafo de escena. Todos estos nodos están dispuestos en un orden jerárquico como se muestra a continuación.

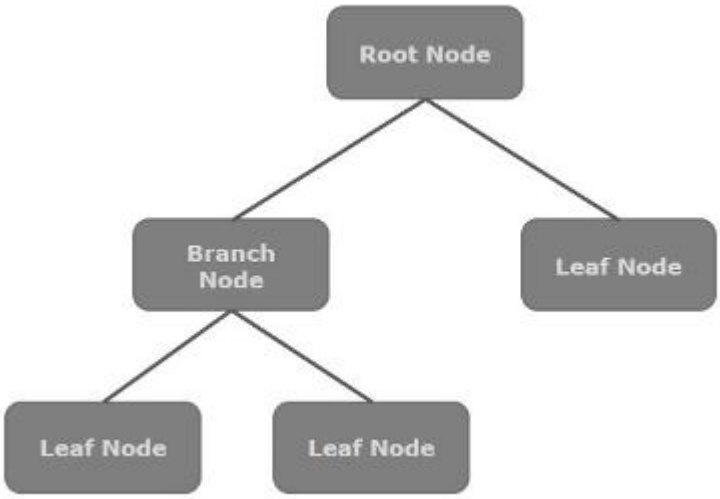

Figura 2: Orden jerárquico de los nodos en Java FX. Fuente: tutorialspoint.com.

Cada nodo del grafo de escena tiene un único padre, y el nodo que no contiene ningún padre se conoce como nodo raíz. Del mismo modo, cada nodo tiene uno o más hijos, y el nodo sin hijos se denomina nodo hoja; un nodo con hijos se denomina nodo rama.

Un nodo sólo puede añadirse una vez a un gráfico de escena. Los nodos de un gráfico de escena pueden tener Efectos, Opacidad, Transformaciones, Manejadores de Eventos, Manejadores de Eventos, Estados Específicos de Aplicación.

JavaFX Graphics Engine

El JavaFX Graphics Engine es el motor interno que se encarga de renderizar todo lo visual en una aplicación JavaFX: desde elementos de la interfaz gráfica hasta gráficos en 2D y 3D, animaciones y efectos visuales. Es una parte fundamental de la arquitectura de JavaFX y permite que las interfaces se vean suaves, modernas y dinámicas.
Es el subsistema gráfico de JavaFX responsable de:

- Renderizar los nodos del Scene Graph.
- Aplicar transformaciones (rotación, escala, traslación).
- Dibujar formas, texto, imágenes y vídeo.
- Manejar efectos gráficos (sombras, desenfoques, reflejos).
- Ejecutar animaciones fluidas.
- Renderizar gráficos 3D.

Está formado por librerías como Prism, GWT, Quantum Toolkit, WebView, Media Engine.
El núcleo del JavaFX Graphics Engine se llama Prism. Prism se encarga de traducir la escena lógica (Scene Graph) en píxeles visibles en pantalla, usando hardware gráfico (GPU) si es posible.

GWT proporciona servicios para gestionar ventanas, temporizadores, superficies y colas de eventos. GWT conecta la plataforma JavaFX con el sistema operativo nativo.

Con JavaFX, también puede incrustar contenido HTML en un gráfico de escena. WebView es el componente de JavaFX que se utiliza para procesar este contenido. Utiliza una tecnología llamada Web Kit, que es un motor interno de navegador web de código abierto. Este componente es compatible con diferentes tecnologías web como HTML5, CSS, JavaScript, DOM y SVG.

El motor multimedia de JavaFX se basa en un motor de código abierto conocido como Streamer. Este motor multimedia admite la reproducción de contenidos de vídeo y audio.

Estructura de una JavaFX Application

En general, una aplicación JavaFX tendrá tres componentes principales: Stage, Scene y Nodes, como se muestra en el siguiente diagrama.

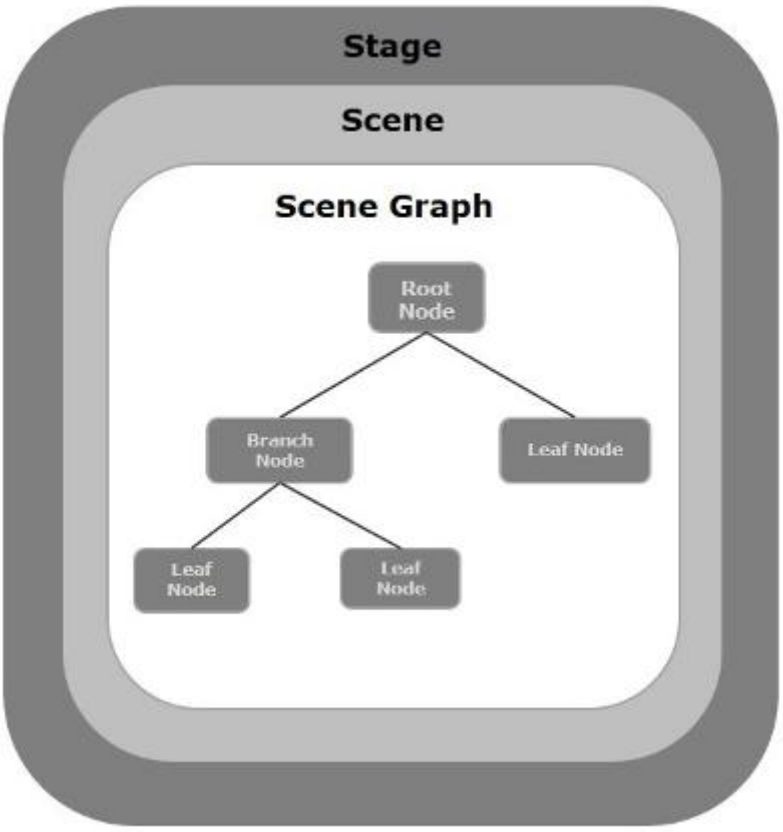

Figura 3: Estructura de una aplicación en Java FX. Fuente: tutorialspoint.com.

Stage

Un stage (una ventana) contiene todos los objetos de una aplicación JavaFX. Está representado por la clase Stage del paquete javafx.stage. El stage primario es creado por la propia plataforma. El objeto stage creado se pasa como argumento al método start() de la clase Application (explicado en la siguiente sección).

Un escenario tiene dos parámetros que determinan su posición: Anchura y Altura. Se divide en Área de contenido y Decoraciones (Barra de título y Bordes).

Hay cinco tipos de escenarios disponibles:

- Decorado.
- No decorado.
- Transparente.
- Unificado.
- Utilitario

Debe llamar al método show() para mostrar el contenido de un escenario.

Scene

Una escena representa el contenido físico de una aplicación JavaFX. Contiene todo el contenido de un gráfico de escena. La clase Scene del paquete javafx.scene representa el objeto escena. En una instancia, el objeto escena se añade a un solo escenario.

Puede crear una escena instanciando la Clase Scene. Puede optar por el tamaño de la escena pasando sus dimensiones (altura y anchura) junto con el nodo raíz a su constructor.

Un grafo de escena es una estructura de datos en forma de árbol (jerárquica) que representa el contenido de una escena. En cambio, un nodo es un objeto visual/gráfico de un gráfico de escena. Un nodo puede incluir

- Objetos geométricos (gráficos) (2D y 3D) como
- Círculo, Rectángulo, Polígono, etc.
- Controles de interfaz de usuario como
- Botón, Casilla de verificación, Cuadro de elección, Área de texto, etc.

- Contenedores (paneles de diseño) como panel de bordes, panel de cuadrícula, panel de flujo, etc.
- Elementos multimedia como objetos de audio, vídeo e imagen.

La clase Node del paquete javafx.scene representa un nodo en JavaFX, esta clase es la superclase de todos los nodos.

Como ya se ha dicho, un nodo puede ser de tres tipos:

- Nodo Raíz - El primer gráfico de escena se conoce como nodo raíz.

- Nodo Rama / Nodo Padre - El nodo con los nodos hijos se conocen como nodos rama / padre. La clase abstracta llamada Parent del paquete javafx.scene es la clase base de todos los nodos padre, y esos nodos padres serán de los siguientes tipos:
    - Grupo - Un nodo grupo es un nodo colectivo que contiene una lista de nodos hijos. Siempre que se renderiza el nodo grupo, todos sus nodos hijos se renderizan en orden. Cualquier transformación o efecto aplicado al grupo se aplicará a todos los nodos hijos.
    - Region - Es la clase base de todos los Controles UI basados en Nodos JavaFX, como Chart, Pane y Control.
    - WebView - Este nodo gestiona el motor web y muestra su contenido.

- Nodo Hoja - El nodo sin nodos hijos se conoce como nodo hoja. Por ejemplo, Rectángulo, Elipse, Caja, ImageView, MediaView son ejemplos de nodos hoja.

Es obligatorio pasar el nodo raíz al gráfico de la escena. Si se pasa el Grupo como raíz, todos los nodos se recortarán a la escena y cualquier alteración en el tamaño de la escena no afectará al diseño de la misma.

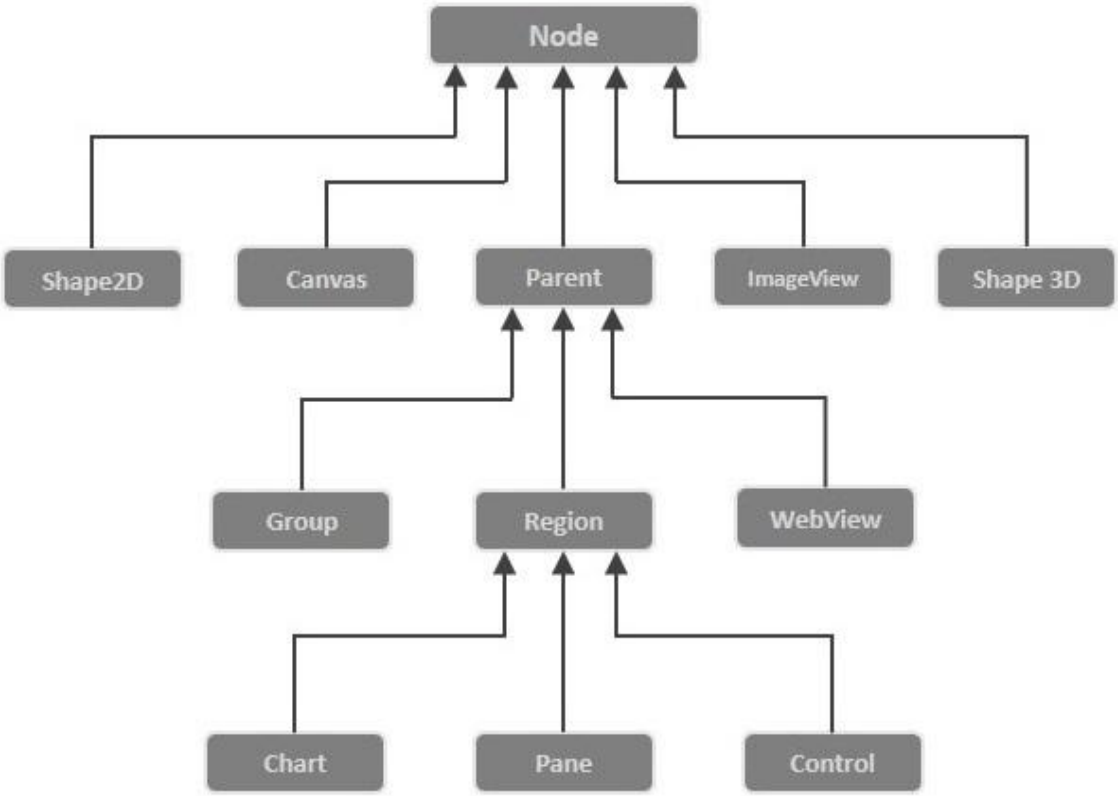

Figura 4: Diagrama de la jerarquía de clases de nodos de JavaFX. Fuente: tutorialspoint.com.

Crear una aplicación Java FX

Para crear una aplicación JavaFX, es necesario instanciar la clase Application e implementar su método abstracto start(). En este método escribiremos el código de la aplicación JavaFX.

La clase Application del paquete javafx.application es el punto de entrada de la aplicación en JavaFX. Para crear una aplicación JavaFX, es necesario heredar esta clase e implementar su método abstracto start(). En este método, necesita escribir todo el código para los gráficos JavaFX En el método main, tiene que lanzar la aplicación usando el método launch(). Este método llama internamente al método start() de la clase Application como se muestra en el siguiente programa.

```
public class JavafxSample extends Application {
   @Override
   public void start(Stage primaryStage) throws Exception {
      /*
      Code for JavaFX application.
```

```
            (Stage, scene, scene graph)
            */
        }
        public static void main(String args[]){
            launch(args);
        }
    }
```

Dentro del método start(), para crear una aplicación JavaFX típica, es necesario seguir los pasos que se indican a continuación:

- Preparar un gráfico de escena con los nodos necesarios.
- Preparar una escena con las dimensiones requeridas y añadirle el grafo de escena (nodo raíz del grafo de escena).
- Prepara un escenario y añade la escena al escenario y muestra el contenido del escenario.

Preparar el Gráfico de Escena y el Stage

Según su aplicación, necesita preparar un gráfico de escena con los nodos requeridos. Dado que el nodo raíz es el primer nodo, necesita crear un nodo raíz. Como nodo raíz, puede elegir entre Group, Region o WebView.

Un Stage es el contenedor de cualquier aplicación JavaFX y proporciona una ventana para la aplicación. Está representado por la clase Stage del paquete javafx.stage. Un objeto de esta clase se pasa como parámetro del método start() de la clase Application.

Con este objeto se pueden realizar diversas operaciones en el escenario. Principalmente puede realizar lo siguiente:

- Establecer el título del escenario utilizando el método setTitle().
- Adjuntar el objeto escena al escenario utilizando el método setScene().
- Muestra el contenido de la escena utilizando el método show() como se muestra a continuación.

```
//Setting the title to Stage.
primaryStage.setTitle("Sample application");
```

```
//Setting the scene to Stage
primaryStage.setScene(scene);

//Displaying the stage
primaryStage.show();
```

Ciclo de vida de una aplicación JavaFX

La clase JavaFX Application tiene tres métodos de ciclo de vida, que son:

- start() - El método de punto de entrada donde el código de gráficos JavaFX se va a escribir.
- stop() - Un método vacío que puede ser sobrescrito, aquí se puede escribir la lógica para detener la aplicación.
- init() - Un método vacío que puede ser sobrescrito, pero no se puede crear escenario o escena en este método.

Además de estos, proporciona un método estático llamado launch() para lanzar la aplicación JavaFX.

Dado que el método launch() es estático, es necesario llamarlo desde un contexto estático (main generalmente). Siempre que se lance una aplicación JavaFX, se llevarán a cabo las siguientes acciones (en el mismo orden).

Se crea una instancia de la clase de aplicación.

- Se llama al método Init().
- Se llama al método start().
- El lanzador espera a que la aplicación termine y llama al método stop().

Terminación de la aplicación JavaFX

Cuando se cierra la última ventana de la aplicación, la aplicación JavaFX se termina implícitamente. Puede desactivar este comportamiento pasando el valor booleano False al método estático setImplicitExit() (debe llamarse desde un contexto estático).

Puede terminar una aplicación JavaFX explícitamente utilizando los métodos Platform.exit() o System.exit(int).

Referencias

Apuntes elaborados a partir de la siguiente documentación:

[1] Apuntes Fernando Barber y Ricardo Ferris. Universidad de Valencia.

[2] Apuntes Programación de José Antonio Díaz-Alejo. IES Camp de Morvedre.

[3] Apuntes de Programación de Carlos Cacho y Raquel Torres. Ceedcv.

[4] Apuntes de Programación Edix Digital Workers.

[5] Pagina web: https://www.tutorialspoint.com/javafx/index.htm

Licencia

CC BY-NC-SA 3.0 ES **Reconocimiento - No Comercial - Compartir Igual (by-nc-sa)**
No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.
NOTA: Esta es una obra derivada de la original realizada por Carlos Cacho y Raquel Torres.